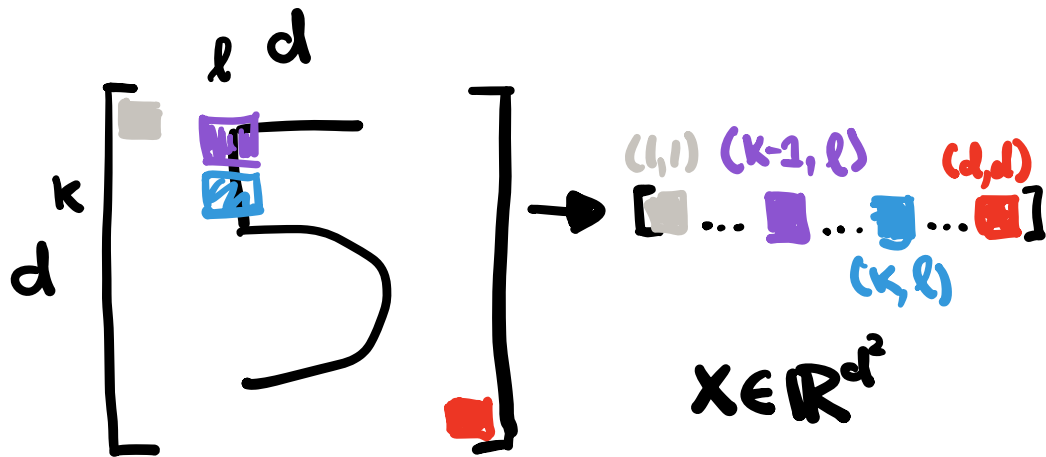
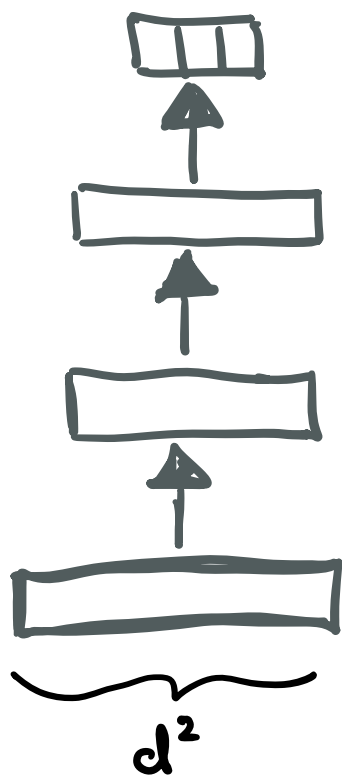


DS 4440

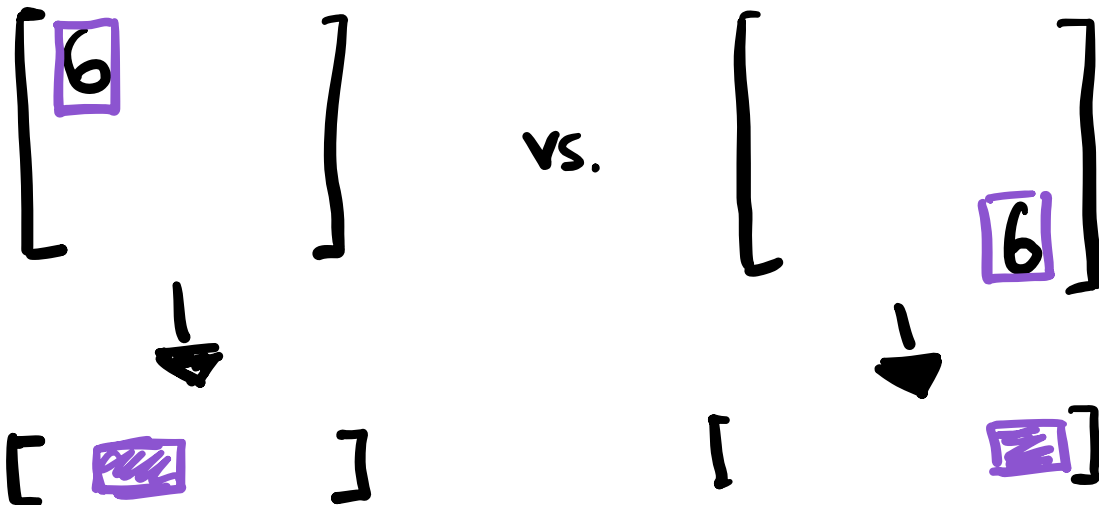
Conv Nets

So far: Simple feed-forward Networks



Problem: This discards
Structure

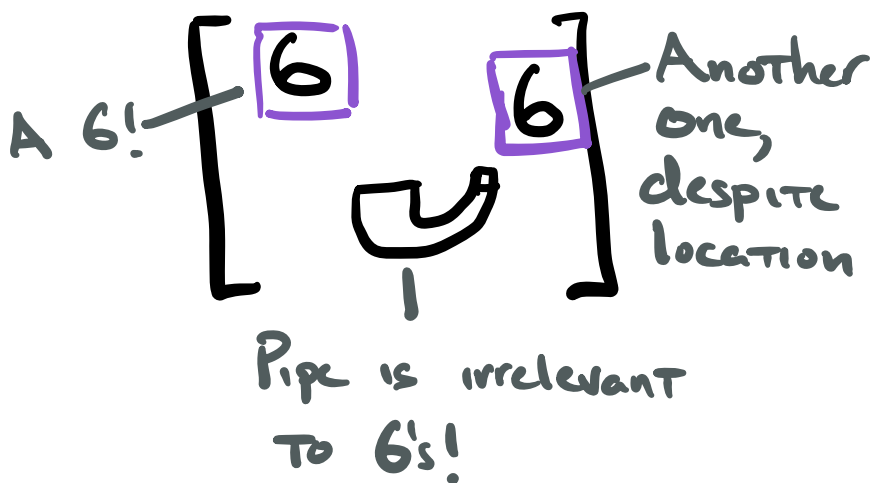
Consider



The parameters associated with these 6's are distinct!

Upshot Model must learn to spot 6 in all locations.

BUT: a 6 is a 6!

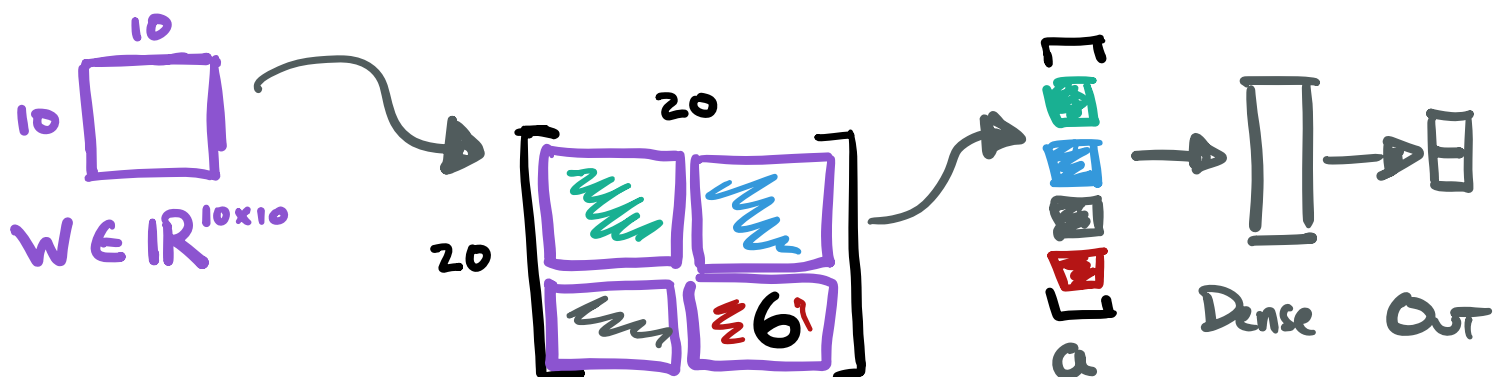


Equivariance
Same detector works, whatever the position.

Invariance
Final prediction doesn't care where 6 appeared.

Enter Convolutional Neural Networks (aka ConvNets aka CNNs)

Basic Idea: Slide Windows  over inputs to yield local features





= local features or activations

Formally assume region A starting at (A_x, A_y) .

$$\boxed{\text{pattern}} = \sigma\left(\sum_{i,j} w_{i+j} \cdot A_{ij}\right)$$

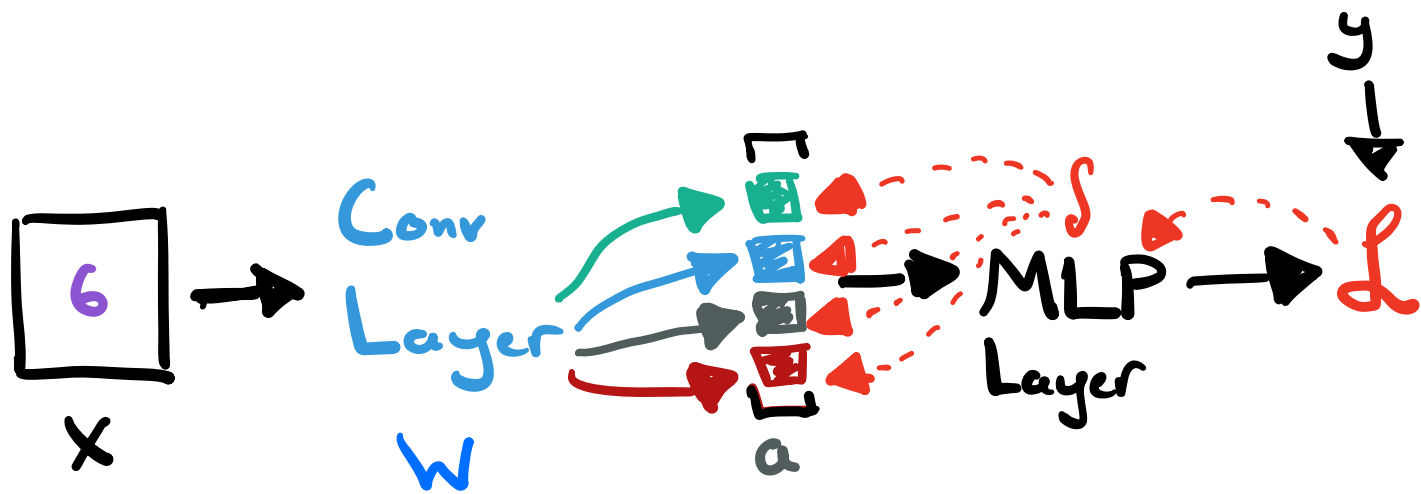
Sum over window size!

$$= \sigma\left(\sum_{i,j} w_{i+j} X_{(A_x+j)(A_y+i)}\right)$$

$$\boxed{\text{pattern}} = \sigma\left(\sum_{i,j} w_{i+j} \cdot B_{ij}\right)$$

The key is that weights w are shared across regions. w is called the kernel or filter.

Q What about  here?



$$\nabla_{\mathbf{W}} \mathcal{L} = \sum_{a_k \in \mathbf{a}} \left(\nabla_{\mathbf{W}} a_k \cdot \frac{\partial \mathcal{L}}{\partial a_k} \right)$$

A concrete example

* Adapted from
"Deep Learning"
by Goodfellow

$$\begin{matrix} & 4 \\ 3 & \begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \end{bmatrix} \\ & \text{Input} \end{matrix} \otimes \begin{matrix} & 2 \\ 2 & \begin{bmatrix} w & x \\ y & z \end{bmatrix} \\ & \text{Kernel} \end{matrix}$$

$$\rightarrow \begin{matrix} & 3 \\ 2 & \begin{bmatrix} w \cdot a + x \cdot b + y \cdot e + z \cdot f & w \cdot b + x \cdot c + y \cdot f + z \cdot g & w \cdot c + x \cdot d + y \cdot g + z \cdot h \\ w \cdot e + x \cdot f + y \cdot i + z \cdot j & w \cdot f + x \cdot g + y \cdot j + z \cdot k & w \cdot g + x \cdot h + y \cdot k + z \cdot l \end{bmatrix} \\ \text{Activation} \end{matrix}$$

In The Simplest Case - as here -

Output Size is

$$(X_h - W_h + 1, X_w - W_w + 1)$$

Input height Kernel height Input width Kernel width

Padding

P	P	P	P	P	P
P	a	b	c	d	P
P	e	f	g	h	P
P	i	j	k	l	P
P	P	P	P	P	P

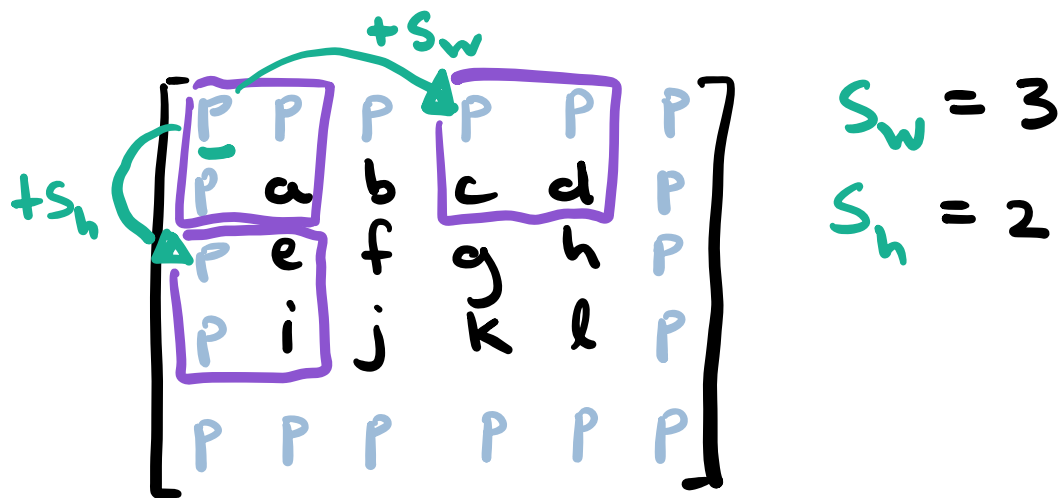
Here we have

Padded height & width by 1.

→ Output Shape

$$(X_h + P_h - W_h + 1, X_w + P_w - W_w + 1)$$

Stride Size has to do with how we pass kernels over inputs - have assumed size of 1 above.

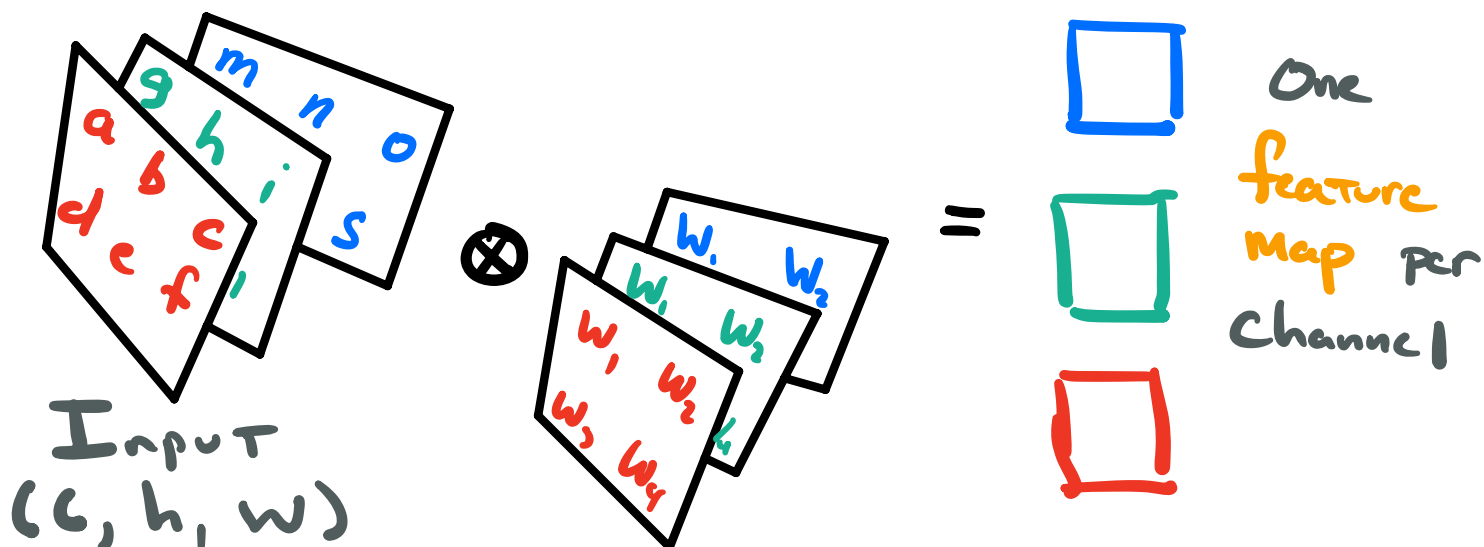


Also affects **Output Size**

$$\left(\frac{(X_h + P_h - W_h + S_h)}{S_h}, \frac{(X_w + P_w - W_w + S_w)}{S_w} \right)$$

Higher Dimensions

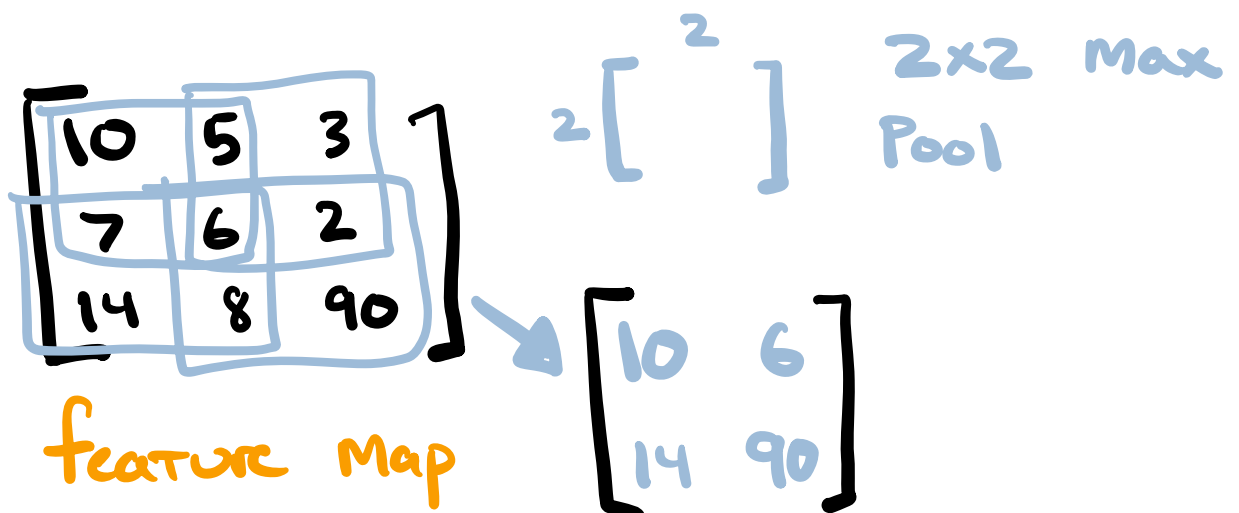
Above we assumed 2d inputs but we can extend to arbitrary Tensors. Useful, e.g. for **RGB** images.



Can Sum for final **map**, or use another **pooling** strategy.

feature map $\rightarrow$ Pool $\rightarrow$ Smaller (often scalar) **Output**

Max pooling extract max value



Q What happens to  Through **Pooling (Max)** operation?

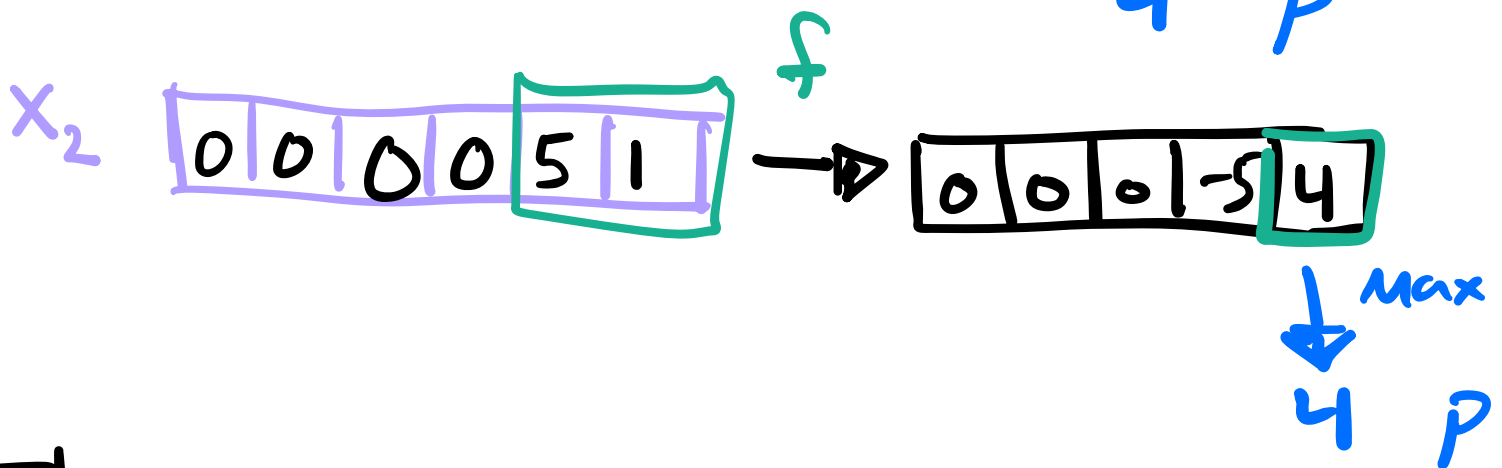
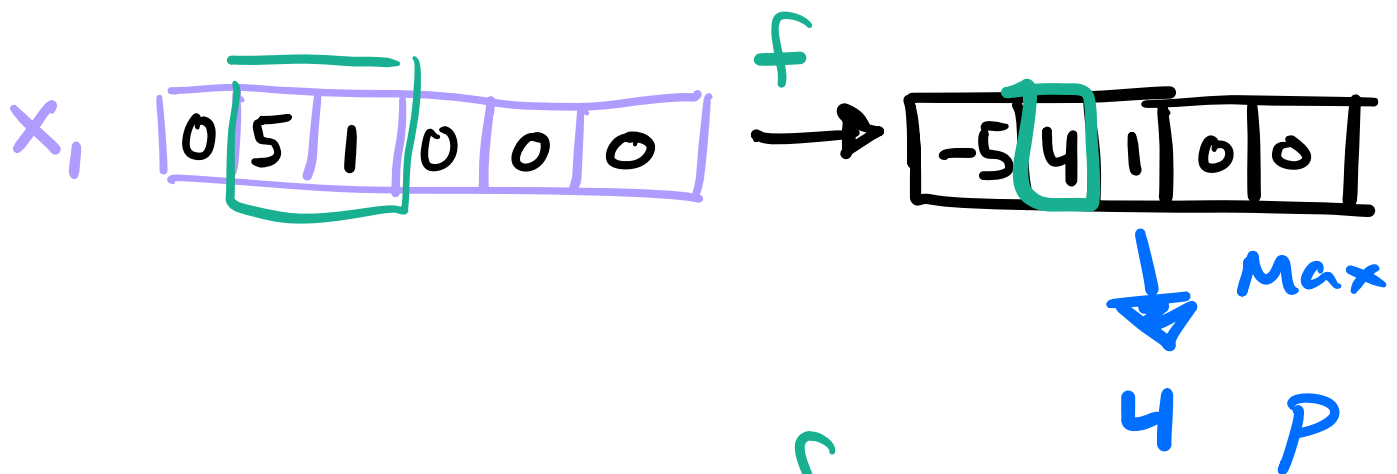
Intuition Individual filters may specialize at recognizing something (Say, cats) $\rightarrow$ **Max pooling** is like asking "does this window contain a cat?" repeatedly **(More on HW4!)**

Example of Invariance

1-d "images" of six pixels.

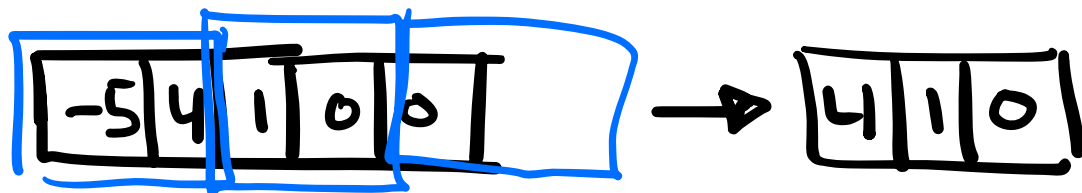
Bright pixel followed by a dim one. (cat ear?)

$f = [1 \ -1]$ responds to this



This is max pooling but we can also do local pooling by sliding

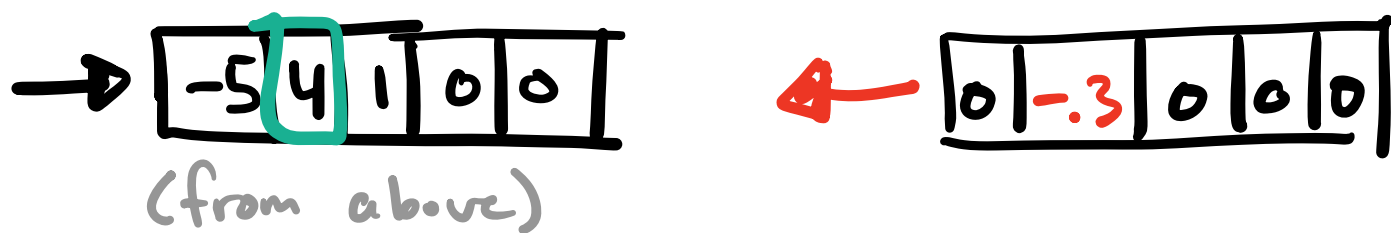
a window



What happens to $\nabla_{\mathcal{L}}$?

Suppose $\frac{\partial \mathcal{L}}{\partial p} = -.3$

This goes back through the **max** dim:



Now Through The Conv **f**:

We have a Sum over

$$\langle \text{incoming} \rangle \cdot \langle \text{patch} \rangle$$

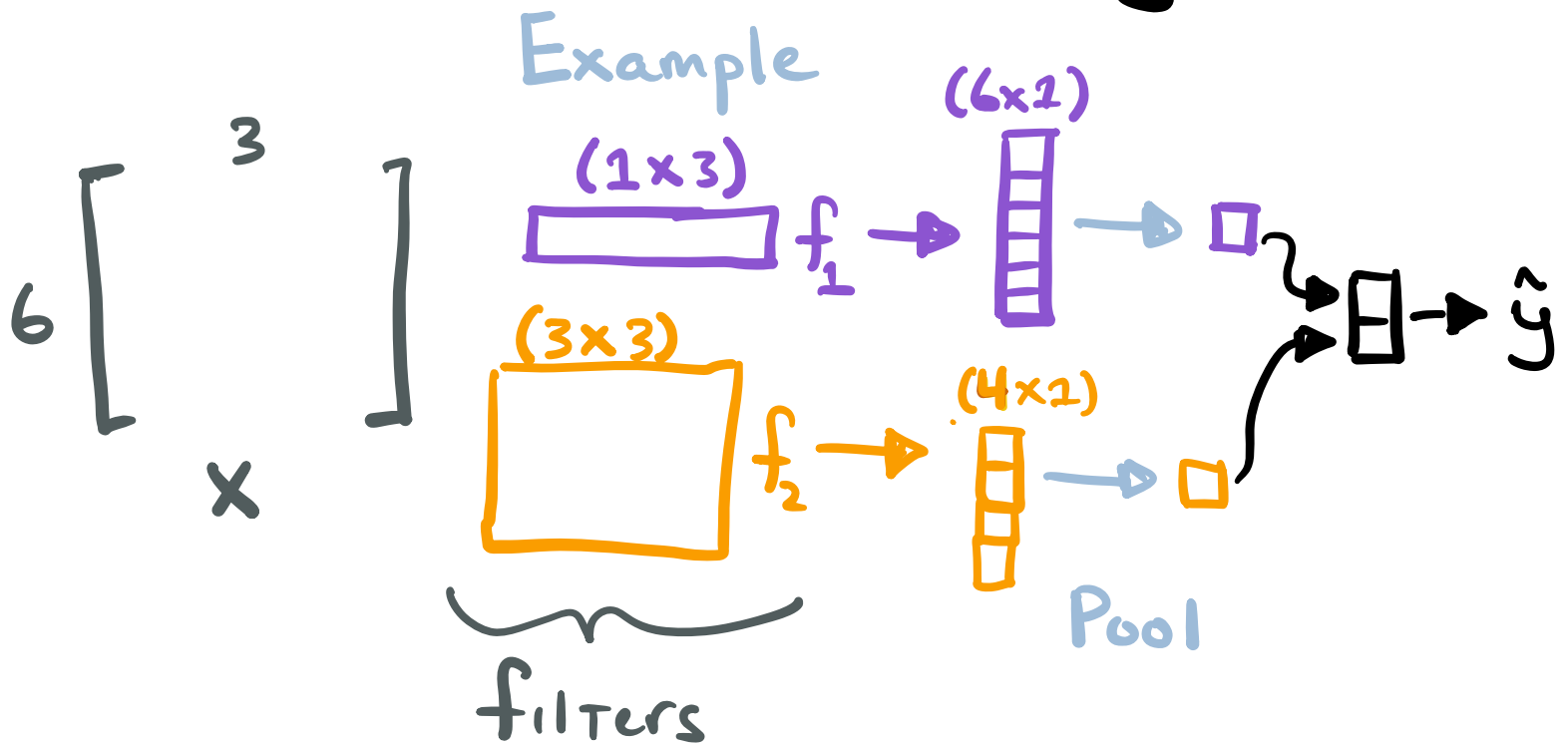
for all patches; but ∇ is nonzero only at index 2. So

$$\frac{\partial \mathcal{L}}{\partial f} = -.3 \cdot [5 \ 1] = [-1.5 \ -.3]$$

Stepping w/ $\alpha = .1$ $f: [1 \ -1] \rightarrow [1.15 \ -.97]$

This increases response to Cat ear ($[5 \ 1]$).

Often we define multiple independent filters, with same or varying size.



Concretely, Consider:

$$\begin{bmatrix} 5 & 3 & 1 \\ 6 & 8 & 1 \\ 2 & 3 & 4 \end{bmatrix} X$$

- No padding
- Two (1×2) filters
- Max pooling (1×2)

$$\begin{bmatrix} 6 & 8 \end{bmatrix} f_1 \quad \begin{bmatrix} -2 & 3 \end{bmatrix} f_2$$

$$\begin{bmatrix} 6 \cdot 5 + 8 \cdot 3 & 6 \cdot 3 + 8 \cdot 1 \\ 6 \cdot 6 + 8 \cdot 8 & 6 \cdot 8 + 8 \cdot 2 \\ 6 \cdot 2 + 8 \cdot 3 & 6 \cdot 3 + 8 \cdot 4 \end{bmatrix} = \begin{bmatrix} 54 & 26 \\ 100 & 56 \\ 36 & 50 \end{bmatrix} \rightarrow \begin{bmatrix} 54 \\ 100 \\ 50 \end{bmatrix}$$

feature map 1

Pooling
(1x2)

$$\begin{bmatrix} -2 \cdot 5 + 3 \cdot 3 & -2 \cdot 3 + 3 \cdot 1 \\ -2 \cdot 6 + 3 \cdot 8 & -2 \cdot 8 + 3 \cdot 1 \\ -2 \cdot 2 + 3 \cdot 3 & -2 \cdot 3 + 3 \cdot 4 \end{bmatrix} = \begin{bmatrix} -1 & -3 \\ 12 & -13 \\ 5 & 6 \end{bmatrix} \rightarrow \begin{bmatrix} -1 \\ 12 \\ 6 \end{bmatrix}$$

$$\oplus \rightarrow [54 \ 100 \ 50 \ -1 \ 12 \ 6]$$